Table of Contents

Displaying Text using `print()` function

```
print("Hello World")
print('My name is Saihan')
```

```
Hello World
My name is Saihan
```

```
print(17+3)
print(10*5-1/5*4+3)
```

```
20
52.2
```

Variables in Python

Suppose we want to display the addition, subtraction, multiplication, and division of two numbers

```
print(90+10)
print(90-10)
print(90*10)
print(90/10)
```

```
print(20+5)
print(20-5)
print(20*5)
print(20/5)
```

! Do you see the problem?

Every time we want to change the numbers, we must write four new lines again and again. This is repetitive and inefficient.

✓ This is where variables help!

A variable is simply a name that stores data. Once values are stored in variables, we can reuse them anytime.

```
a = 80
b = 10

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Here `a` and `b` are variables. Now, if you want to calculate using different numbers, you only need to change `a` and `b`.

Data Type	Python Name	What it means	Example
Integers	int	Whole numbers (no decimal point)	`5`, `-3`, `0`, `1000`
Floats	float	Numbers with decimal points	`3.14`, `-0.5`, `100.0`, `2.0`
Strings	str	Text (anything inside quotes)	`"Hello"`, `'Python'`, `"2026"`
Booleans	bool	True or False values	`True`, `False`, `5 > 3` → `True`

```
a = 30
b = 15

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

```
45
15
450
2.0
```

```
name = "Abdullah"
print(name)
```

```
Abdullah
```

```
country = "Bangladesh"
print(country)
```

```
Bangladesh
```

```
age = 26
print(age)
```

```
26
```

```
score = 9.5
print(score)
```

```
9.5
```

```
name = "Abdullah Saihan"
city = "Dhaka"

print("Hello, My name is ", name)
print("I'm from ", city)
```

```
Hello, My name is  Abdullah Saihan
I'm from  Dhaka
```

Basic Data Types

In Python, data can be of different types. For now, we will learn the 4 most common types:

1. String (str) — Text Data

A string is used to store text.

You can write a string inside:

- double quotes → `"Hello"`
- single quotes → `'Python'`

Examples:

```
name = "Saihan"
city = 'Dhaka'
```

Note that nything inside quotes becomes text, even numbers -

```
age = "25"   # This is a string, not a number
```

2. Integer (int) — Whole Numbers

Integers are numbers without decimal points.

Examples:

```
age = 25
year = 2025
count = -5
```

3. Float (float) — Decimal Numbers

Floats are numbers with decimal points.

Examples:

```
price = 10.5
height = 5.9
temperature = -2.75
```

4 .Boolean (bool) — True or False

A Boolean stores either:

`True` or `False`

Examples:

```
is_active = True
is_little = False
```

Booleans often come from comparisons:

```
print(5 > 3)    # True
print(10 < 7)   # False
print(7 == 6)   # False
print(6 != 7)   # True
print(2 >= 5)   # False
print(6 <= 9)   # True
```

```
age = 10
temparature = 34.5


name = "Saihan"
city = "Dhaka"


is_rainy = False
is_sunny = True



print(type(age))
print(type(temparature))
print(type(name))
print(type(city))
print(type(is_rainy))
print(type(is_sunny))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'str'>
<class 'bool'>
<class 'bool'>
```

```
result = 4 > 5

print(result)
print(type(result))
```

```
False
<class 'bool'>
```

```
## comparison operators
## boolean data often comes from comarison

print(5 > 3)    # 5 greater than 3 -> True
print(10 < 7)   # 10 less than 7 -> False
print(7 == 6)   # 7 equal to 6 -> False
print(6 != 7)   # 6 not equal to 7 -> True
print(2 >= 5)   # 2 greater than or equal to 5 -> False
print(6 <= 9)   # 6 less than or equal to 9 -> True
```

```
True
False
False
True
False
True
```

Commenting in Python

Comments are notes in your code that are ignored by Python when the program runs. They are used to explain code, make it readable, or temporarily disable code.

a) Inline comment

```
x = 10  # Assign 10 to variable x
print(x)  # Print the value of x
```

b) Single-line comment

```
# This is a single-line comment
print("Hello, Python!")
```

c) Multi-line comment

```
# This is a multi-line comment
# explaining what the code does
# line by line
greet = "Hi"
name = "Zami"
print(f"{greet} {name}")
```

d) Commenting out code to temporarily prevent it from running

```
# The following code cannot be run because it is commented out.

# greet = "Hi"
# name = "Zami"
# print(f"{greet} {name}")
```

```
# This is a single-line comment
print("Welcome to Python!")  # Inline comment: prints welcome message


# This is a multi-line comment
# explaining the next few lines of code

greet = "Hello"
name = "Zami"
print(f"{greet} {name}")

# Commenting out code to temporarily disable it
# age = int(input("Enter your age: "))
# print("Your age is", age)
```

```
Welcome to Python!
Hello Zami
```

Type Conversion

Type conversion means changing one data type into another.

Example: converting a string into an integer.

Common conversions:

```
int("10")       # string → integer
float("10.5")   # string → float
str(25)         # integer → string
str(3.14)       # float → string
```

`type()` Function

type() tells you what data type a value or variable is.

Examples:

```
print(type("Hello"))   # str
print(type(10))        # int
print(type(10.5))      # float
print(type(True))      # bool
```

`"10"` is a String, NOT a Number

Even though `"10"` looks like a number, because it's inside quotes, Python treats it as text (string).

```
value = "10"
print(type(value))   # <class 'str'>
```

You cannot do math with it unless you convert it:

```
a = "10"
b = 5

print(a + b)   # ❌ Error
```

Convert it first:

```
print(int(a) + b)   # 15
```

```
a = "10"        # a is a string
b = int(a)      # convert to integer

print(a)
print(type(a))
print(b)
print(type(b))
```

```
10
<class 'str'>
10
<class 'int'>
```

```
x = "10.5"      # x is a string
y = float(x)    # convert to float

print(x)
print(type(x))
print(y)
print(type(y))
```

```
10.5
<class 'str'>
10.5
<class 'float'>
```

```
num = 25          # num is a integer
text = str(num)   # convert to string

print(num)
print(type(num))
```

```
print(text)
print(type(text))
```

```
25
<class 'int'>
25
<class 'str'>
```

```
pi = 3.14           # pi is a float
pi_text = str(pi)   # convert to string

print(pi)
print(type(pi))
print(pi_text)
print(type(pi_text))
```

```
3.14
<class 'float'>
3.14
<class 'str'>
```

Arithmatic and Comarison Operators

Arithmatic Operators

Operator	Name	What it does (Simple)	Example	Result
+	Addition	Adds two numbers	5 + 3	8
-	Subtraction	Subtracts one number from another	5 - 3	2
*	Multiplication	Multiplies two numbers	5 * 3	15
/	Division	Divides and returns float	5 / 2	2.5
//	Floor Division	Divides and returns whole number (no decimal)	5 // 2	2
%	Modulus	Returns remainder of division	5 % 2	1
**	Exponent (Power)	Raises number to a power	2 ** 3	8

Quick notes:

- `/` always returns a decimal (float)
- `//` removes the decimal part
- `%` is useful for checking even/odd numbers
- `**` means power (like 2**3 = 2^3)

Comarison Operators

Operator	Name	What it does (Simple)	Example	Result
==	Equal to	Checks if two values are equal	5 == 5	True
!=	Not equal to	Checks if two values are different	5 != 3	True
>	Greater than	Checks if left is bigger than right	5 > 3	True
<	Less than	Checks if left is smaller than right	5 < 3	False
>=	Greater than or equal	Bigger OR equal	5 >= 3	True

Operator	Name	What it does (Simple)	Example	Result
<=	Less than or equal	Smaller OR equal	5 <= 3	False

Quick notes:

- These operators always return True or False
- Used in if conditions, loops, and decision-making

```
a = 50
b = 10

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

```
60
40
500
5.0
```

```
print(7/2) # normal division
```

```
3.5
```

```
print(7//2)  # floor division
```

```
3
```

```
a = 11
b = 3

print(a % b)
```

```
2
```

```
a = 3
b = 4

print(a**b)
print(3**4)
print(3*3*3*3)
```

```
81
81
81
```

```
print(5 > 3)    # 5 greater than 3 -> True
print(10 < 7)   # 10 less than 7 -> False
print(7 == 6)   # 7 equal to 6 -> False
print(6 != 7)   # 6 not equal to 7 -> True
print(2 >= 5)   # 2 greater than or equal to 5 -> False
print(6 <= 9)   # 6 less than or equal to 9 -> True
```

```
True
False
False
True
False
True
```

Taking User Input in Python

In Python, we use the `input()` function to take data from the user.

Basic Input

```
name = input()
```

This will wait for the user to type something and press Enter. Whatever the user types is stored in the variable name.

Input with a Message (Prompt)

We usually show a message so the user knows what to type.

```
name = input("Enter your name: ")
```

Example:

```
Enter your name: Saihan
```

Then `name` variable will store "Saihan".

Important: input() Always Returns a String

Even if the user types a number, Python treats it as string (text).

Example:

```
age = input("Enter your age: ")
print(type(age))   # str
```

If you want to use the input as a number, convert it:

```
age = int(input("Enter your age: "))
```

or for decimal numbers:

```
price = float(input("Enter the price: "))
```

```
username = input("Please enter your username: ")
password = input("Please enter your password: ")
```

```
age = int(input("Please enter your age: "))
```

```
Please enter your username:  saihan
Please enter your password:  123#456
Please enter your age:  26
```

```
print(username)
print(type(username))
```

```
saihan
<class 'str'>
```

```
print(password)
print(type(password))
```

```
123#456
<class 'str'>
```

```
print(age)
print(type(age))
```

```
26
<class 'int'>
```

```
# input() -> always return strings

a = input("Enter first number: ")
b = input("Enter 2nd number: ")
```

```
Enter first number:  10
Enter 2nd number:  30
```

```
print(a, b)
print(type(a))
print(type(b))
```

```
10 30
<class 'str'>
<class 'str'>
```

String Concatenation and Repeataion

String Concatenation (+)

- Used to **join (add) two or more strings**
- Only works with strings (not numbers directly)

Example:

```
print("Hello" + " " + "World")   # "Hello World"

name = "Sam"
print("Hello " + name)    #  "Hello Sam"
```

String Repetition (*)

- Used to **repeat a string multiple times**
- Multiply a string by a number

Example:

```
print("Hi" * 3)        # "HiHiHi"
print("-" * 5)         # "-----"
```

Important Notes:

- You **cannot** add string and number directly: `"Age: " + 20` ❌ Error
- Convert number to string first: `"Age: " + str(20)` ✅ "Age: 20"

```
first_name = "Abdullah"
last_name = "Saihan"


full_name = first_name + " " + last_name   # string concatenation
print(full_name)

print("="*20)  # string repeatation
```

```
Abdullah Saihan
====================
```

Output formatting using `print()`

The `print()` function is used to display output on the screen. Python provides several ways to format this output neatly and clearly.

Basic `print()`

You can print text, numbers, variables, or expressions.

```
print("Hello Python")  # printing text
print(10)              # printing number
print(5 + 3)           # printing expression
name = "Hasnat"
print(name)            # printing variable
```

Output:

```
Hello Python
10
8
Hasnat
```

Printing multiple items using a single `print()`

You can print more multiple items by separating them with commas.

printing multiple text/numbers

```
print("Hello", "World!")
print("Hello", 1234, "mic", "testing!")
```

Output:

```
Hello World!
Hello 1234 mic testing!
```

printing multiple variables

```
a = 10
b = 20
c = 30
print(a, b, c)
```

Output:

```
10 20 30
```

printing multiple text and variables together

```
name = "Zubayer"
age = 20
print("My name is", name, "I am", age, "years old")
```

Output:

```
My name is Zubayer I am 20 years old
```

`sep` (Separator)

The sep parameter defines how multiple values are separated when printed.

Default separator: `1 space ( ) --> sep=" "`

```
print("10", "05", "2025", sep="-")
print("2025", "05", "2025", sep="/")
```

Output:

```
10-05-2025
10/05/2025
```

Escape Characters `e.g. \n, \t`

Escape characters are special characters used inside strings.

a) New Line `\n`

Moves the cursor to a new line.

```
print("Hello\nWorld")
```

Output:

```
Hello
World
```

b) Tab Space `\t`

Adds horizontal spacing (tab).

```
print("Name\tAge\tCity")
print("Ali\t25\tDhaka")
```

Output:

```
Name    Age     City
Ali     25      Dhaka
```

f-String (Formatted String)

f-strings allow you to insert variables directly inside strings.

```
name = "Saihan"
age = 26
print(f"My name is {name} and I am {age} years old.")
```

Output:

```
My name is Saihan and I am 26 years old.
```

You can also use expressions:

```
a = 10
b = 5
print(f"Sum = {a + b}")
```

Output:

```
Sum = 15
```

Number Formatting (int and float)

a) Integer Formatting

Leading zeros:

```
num = 7
print(f"{num:03d}")
```

Output:

```
007
```

b) Float Formatting

Control decimal places:

```
pi = 3.1415926
print(f"{pi:.2f}")
```

Output:

```
3.14
```

c) Both together

```
price = 45.6789
quantity = 3
print(f"Quantity: {quantity:02d}, Price: {price:.2f}")
```

Output:

```
Quantity: 03, Price: 45.68
```

Printing same thing using multple ways

```
name = "Saihan"
age = 26
```

```
print("Name:", name, "Age:" age)
print(f"Name: {name} Age: {age}")
```

Output:

```
Name: Saihan Age: 26
Name: Saihan Age: 26
```

```
print("10", "07", "2025", sep="-")
print("10-07-2025")
```

Output:

```
10-07-2025
10-07-2025
```

```
a = 10
b = 20
c = 30
print(a, "|", b, "|", c)
print(a, b, c, sep=" | ")
print(f"{a} | {b} | {c}")
```

Output:

```
10 | 20 | 30
10 | 20 | 30
10 | 20 | 30
```

Key Points

- Use comma (,) to print multiple items
- Python automatically adds a space between items
- sep controls how items are separated
- f-strings make output clean and readable

```
a = 10
b = 20
c = 30
print(a, "|", b, "|", c)
print(a, b, c, sep=" | ")
print(f"{a} | {b} | {c}")
```

```
10 | 20 | 30
10 | 20 | 30
10 | 20 | 30
```

```
name = "Saihan"
age = 20
print("Name:", name)
print("Age:", age)
print("Name:", name, "Age:", age)
```

```
Name: Saihan
Age: 20
Name: Saihan Age: 20
```

```
print("10", "01", "2000", sep="/")
print("10/01/2000")
```

```
10/01/2000
10/01/2000
```

```
print("10", "01", "2000", sep="-")
print("10-01-2000")
```

```
10-01-2000
10-01-2000
```

```
name = "Saihan"
age = 20
print("Name:\n", name, "\nAge:\n", age, sep="")
```

```
Name:
Saihan
Age:
20
```

```
name = "Saihan"
age = 20
print("Name:", name, "\tAge:\t", age, sep=" ")
```

```
Name: Saihan    Age:     20
```

```
name = "Saihan"
age = 20
print("Name:\t", name, "\nAge:\t", age, sep=" ")
```

```
Name:    Saihan
Age:     20
```

```
name = "Rayhan"
age = 18
workplace  = "banglalink"

print(f"Hi I'm {name}. \nI'm {age} years \t old, \nCurrenly I'm working at {workplace}")
```

```
Hi I'm Rayhan.
I'm 18 years     old,
Currenly I'm working at banglalink
```

```
si = 7
name = "Saihan"

print(f"{si:03}\t{name}")
```

```
007     Saihan
```

```
price = 10.544342

print(f"Price: {price:.5f}")
```

```
Price: 10.54434
```

```
# f-string


name = "Saihan"
city = "Dhaka"
age = 20


print("Hello, My name is ", name)
print("I'm from ", city)
print("I'm", age, "years old")
```

```
Hello, My name is  Saihan
I'm from  Dhaka
I'm 20 years old
```

```
name = "Saihan"
city = "Dhaka"
age = 20

print(f"Hello, My name is {name}")
print(f"I'm from {city}")
print(f"I'm {age} years old")
```

```
Hello, My name is Saihan
I'm from Dhaka
I'm 20 years old
```

```
print('His name is "Saihan", He is from Dhaka')
print("His name is 'Saihan', He is from Dhaka")


```

```
His name is "Saihan", He is from Dhaka
His name is 'Saihan', He is from Dhaka
```

```
print()
print("Result")
print("="*30)
print(f"a+b = {a + b}")
print(f"a-b = {a - b}")
print(f"{a} x {b} = {a * b}")
```

```
print(f"{a} / {b} = {a / b}")
```

```
Result
==============================
a+b = 30.0
a-b = 10.0
20.0 x 10.0 = 200.0
20.0 / 10.0 = 2.0
```

```
eval("2+3+10-9")
```

```
6
```

Notes on Debugging Errors

Debugging means finding and fixing errors and mistakes in a program. For beginners, these three types of errors occur most often:

- Syntax Error
- Name Error
- Type Error

Syntax Error

A Syntax Error occurs when Python cannot understand the code because it breaks the rules of the language.

a) Missing Quote

```
print("Hello World)
```

Output:

```
SyntaxError: unterminated string literal
```

b) Mixing Single Quote and Double Quote

```
print("Hello World')
```

Output:

```
SyntaxError: unterminated string literal
```

c) Missing Parenthesis

```
print("Hello World"
```

Output:

```
SyntaxError: '(' was never closed
```

Name Error

A Name Error occurs when you use a variable that has not been defined

a) Undefined Variable

```
print(country)
```

Output:

```
NameError: name 'country' is not defined
```

b) Misspelled Variable Name

```
number = 10
print(numbr)
```

Output:

```
NameError: name 'numbr' is not defined
```

c) Using Variable Before Assignment

```
print(place)
place = "Dhanmondi"
```

Output:

```
NameError: name 'place' is not defined
```

Type Error

A Type Error occurs when you perform an operation on incompatible data types.

a) Unsupported Operand Type

```
num1 = 10
num2 = "5"  # notice that num2 is not actually a number, it is a string
print(num1 + num2)
```

Output:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
a = "20"
b = "10"
print(a - b)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[214], line 3
      1 a = "20"
      2 b = "10"
----> 3 print(a - b)

TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

```
a = "20"
b = "10"
print(int(a) - int(b))
```

```
10
```

```
num1 = 10
num2 = "5"
print(num1 + num2)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[216], line 3
      1 num1 = 10
      2 num2 = "5"
----> 3 print(num1 + num2)

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
num1 = 10
num2 = "5"
print(num1 + int(num2))
```

```
15
```

```
print('Hello World")
```

```
  Cell In[218], line 1
    print('Hello World")
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

```
print("Hello World)
```

```
  Cell In[219], line 1
    print("Hello World)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

```
print("Hello World"
```

```
  Cell In[220], line 1
    print("Hello World"
                       ^
SyntaxError: incomplete input
```

```
print("Hello World")
```

```
Hello World
```

```
name = "Saihan"

print(f"Hello {naem}")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[223], line 3
      1 name = "Saihan"
----> 3 print(f"Hello {naem}")

NameError: name 'naem' is not defined
```

```
name = "Saihan"

print(f"Hello {name}")
```

```
Hello Saihan
```

```
num1 = 10
num2 = "3"

print(num1 + num2)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[143], line 4
      1 num1 = 10
      2 num2 = "3"
----> 4 print(num1 + num2)

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
birth_year = input("Enter your birth year: ")
age = 2025 - birth_year

print(f"User is {age} years old")
```

```
Enter your birth year:  2000
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[225], line 2
      1 birth_year = input("Enter your birth year: ")
----> 2 age = 2025 - birth_year
      4 print(f"User is {age} years old")

TypeError: unsupported operand type(s) for -: 'int' and 'str'
```

```
birth_year = int(input("Enter your birth year: "))
age = 2025 - birth_year

print(f"User is {age} years old")
```

```
Enter your birth year:  2000
User is 25 years old
```

```
num1 = 10
num2 = 0

print(num1+num2)
print(num1/num2)
```

```
10
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[227], line 5
      2 num2 = 0
      4 print(num1+num2)
----> 5 print(num1/num2)

ZeroDivisionError: division by zero
```

Project: Simple Calculator

```
a = float(input("Enter 1st number: "))
b = float(input("Enter 2nd number: "))

print()
print("Result")
print("="*30)
print(f"Addition = {a + b}")
print(f"Subtration = {a - b}")
print(f"Multiplication = {a * b}")
print(f"Division = {a / b}")
```

```
Enter 1st number:  10
Enter 2nd number:  5

Result
============================
Addition = 15.0
Subtration = 5.0
Multiplication = 50.0
Division = 2.0
```